

Sri Lanka Institute of Information Technology



IE3132 - Penetration Testing
Year 3, Semester 1- 2026

Project Report

Assignment 01 - CTF Play Box Implementation

Student ID	Name
IT24102800	RAJAPAKSHA K.R.D.A.P.M.
IT24102481	JAYASOORIYA N.D.K.P.
IT24102454	JAYASUNDARA J.M.M.N.
IT24104202	MANMITHA K.G.I.

09/22/2026

SUMMARY

This report presents the design and planning phase of **Cybervault: Project Null Byte**, a web-based capture the flag platform developed for cybersecurity skill development. The proposed platform contains 6 challenges covering OSINT, Steganography, Cryptography, Digital Forensics, Git Forensics, Reverse Engineering, Linux, Web Security Domains Programming and Scripting.

The CTF is built around a realistic investigative scenario in which participants assume the role of a penetration tester tracing a simulated data breach across multiple systems and artefacts. Each stage is designed to provide practical, hands-on learning experiences through realistic attacks and investigation scenarios, while maintaining structured difficulty progression from easy to expert. The platform is intended for undergraduate cybersecurity students and beginner-to-intermediate participants seeking to strengthen their practical security skills.

The proposed architecture isolated challenge environments, a central flag validation service, and a web-based scoreboard for tracking participant progress. This document details the platform architecture, stage specifications, technology stack, deployment and isolation plan, testing strategy, and individual team contributions to support the subsequent implementation phase. The design priorities technical feasibility, safe isolation, and clear alignment with the module's learning outcomes.

OBJECTIVES

1. **Design a realistic CTF environment** that simulates practical cybersecurity scenarios in a controlled, isolated setting.
2. **Develop 6 cybersecurity challenges** that advance from easy to hard difficulty, building participant skills incrementally.
3. **Cover multiple cybersecurity domains** including OSINT, Steganography, Cryptography, Digital Forensics, Git Forensics, Reverse Engineering, Linux, Web Security Domains Programming and Scripting to ensure broad practical exposure.
4. **Improve practical cybersecurity skills** through hands-on problem-solving, tool usage, and real-world attack and defense techniques.
5. **Provide practical learning opportunities** that bridge theoretical knowledge with applied penetration testing exercises.
6. **Establish a secure deployment architecture** with proper isolation, network segmentation, and controlled access to prevent unintended impact on institutional or public systems.

CTF CONCEPT & SCENARIO

CyberTech solutions company has experienced a suspected cyber-attack. Several systems within the organization have been compromised and sensitive information may have been exposed.

Participants take the role of junior penetration testers assigned to investigate the incident. They must gather intelligence, analyze evidence, identify security weaknesses, and recover hidden flags through six challenges.

The investigation includes OSINT, Steganography, Cryptography, Digital Forensics, Git Forensics, Reverse Engineering, Linux, Web Security Domains Programming and Scripting investigation.

Field	Details
CTF / Box Name	CyberVault: Project Null Byte
Target Audience	Undergraduate cybersecurity students, beginner to intermediate level and students interested in penetration
Learning Outcomes	Technical skills
Challenge Count	6
Difficulty Level	Easy → Moderate → Hard → Expert
Flag Format	IE3132{...}
Platform Type	Web-based
Operating Environment	Windows
Deployment Method	Local server
Port	3000
Tools / Technologies	Visual Studio Code
Estimated Resources	Processor: Intel Core i5 or equivalent Memory: 16 GB RAM Storage: 20 GB Available Disk Space

LEARNING OUTCOMES

- Apply information gathering and reconnaissance techniques.
- Analyze vulnerabilities in various environments.
- Use cybersecurity tools to investigate security incidents.
- Understand web, network, and system security concepts.
- Apply practical penetration testing methodologies.

SELECTED DOMAINS & DIFFICULTY LEVEL

	Domain	Difficulty Model
Challenge 1	OSINT, Digital Forensics, Git Forensics, Steganography	Hard
Challenge 2	Digital Forensics, Steganography, OSINT	Expert
Challenge 3	Reverse Engineering, Web Security, Programming / Scripting	Hard
Challenge 4	Steganography	Easy
Challenge 5	Cryptography	Moderate
Challenge 6	Linux / System Security	Hard

DOMAIN JUSTIFICATION

The selected domains were chosen to provide participants with exposure to multiple areas of cybersecurity. OSINT introduces reconnaissance techniques, Steganography teaches hidden-data analysis, Cryptography develops decoding and decryption skills, Digital Forensics focuses on evidence recovery, Reverse Engineering enhances code analysis abilities, and Linux / System Security provides practical experience with users, permissions, services, and command-line investigation.

CHALLENGE SPECIFICATIONS

Challenge 1

Field	Information
Stage ID	CTF-OSINT
Title	The Ghost Who Walked
Domain	OSINT / Reconnaissance, Digital Forensics, Git Forensics
Difficulty	Hard
Difficulty Justification	Multi-step investigation requiring OSINT, metadata analysis, decoding, Git forensics, and critical thinking. The trail is non-obvious and requires multiple pivots between different data sources.
Scenario	Investigate a corporate whistleblower known as "NoOne" who leaked evidence of a data breach at Aethelred Industries.
Learning Objective	Apply advanced OSINT techniques, recover hidden information from metadata, conduct Git forensics, and correlate data from multiple sources.
Environment	"Internet" environment with hosted services (image, Pastebin, archived forum, GitHub repository)
Player Task	Follow the digital trail from a single image to find the flag hidden in Git commit history
Tools Required	exiftool, Base64, Git, Web Browser

Challenge 2

Field	Information
Stage ID	CTF-DF
Title	The Ghost's Workstation
Domain	Digital Forensics, Steganography, OSINT
Difficulty	Hard
Difficulty Justification	Multi-step chain: disk recovery → archive extraction → metadata analysis → Base64 decoding → steganography. Requires knowledge of 5+ tools and non-obvious pivots.
Scenario	Investigator seized a workstation disk image named disk_image_IE3132.dd from the whistleblower. The evidence was deleted. Recover it and extract the final flag.
Learning Objective	Apply disk forensics, metadata analysis, encoding, and steganography in a chained investigation.
Environment	Disk image file (disk_image_IE3132.dd)
Player Task	Recover deleted ZIP → extract image → read metadata → decode password → extract hidden flag with steghide
Tools Required	sleuthkit (fls, icat), exiftool, base64, steghide, unzip

Challenge 3

Field	Information
Stage ID	CTF-RE
Title	The Ghost's Game
Domain	Reverse Engineering, Client-Side Analysis, JavaScript Forensics
Difficulty	Hard
Difficulty Justification	Multi-step chain: gameplay → source analysis → variable manipulation → function invocation. Requires understanding of JavaScript runtime, DevTools usage, and logical reasoning. The win condition is non-obvious and hidden behind obfuscated names.
Scenario	PulseForge Studios' game Ghost Runner was trojanized by a rival attacker, SilentPlayer. A hidden flag is embedded in the JavaScript. Recover it to prove tampering.
Learning Objective	Apply client-side reverse engineering to identify and manipulate a JavaScript game's win condition, extracting hidden data via runtime inspection.
Environment	Single HTML file (ghost_runner.html) + background.png, hosted statically on Github.
Player Task	Analyze the game's source, find the win condition, and either, (a) complete Level 4 legitimately. (b) force the win via DevTools Console.
Tools Required	Web browser, DevTools (F12), Text Editor (optional)

Challenge 4

Field	Information
Stage ID	CTF-S
Title	Hidden in Plain Sight
Domain	Steganography
Difficulty	Easy
Difficulty Justification	The participant follows a short and guided investigation path: identify the supplied JPEG, inspect its metadata for a clue, infer the extraction passphrase, and use a standard steganography utility to recover an embedded text artefact.
Scenario	A normal-looking image is recovered during the CyberTech Solutions incident investigation. Investigators suspect that additional information has been concealed inside the file.
Learning Objective	Identify hidden information in image-based evidence; inspect metadata; select an appropriate steganography tool; and recover a concealed artefact.
Challenge Environment	Static JPEG evidence file distributed through the CTF platform. No vulnerable network service is required for this stage.
Player Task	Investigate the image, discover the extraction clue, recover the embedded text file, and submit the stage flag.
Flag Location / Logic	The stage flag is stored inside a text artefact embedded in the JPEG using Steghide. The recovered artefact also contains a keyword required by CTF-S. Exact answers are intentionally omitted from the main report.
Required Tools	Basic file inspection utility, ExifTool, and Steghide (or equivalent authorized tools).
Dependencies	Requires access to the CTF-S evidence image. The output of this stage becomes an input dependency for CTF-S.
Expected Solution Path	Inspect JPEG → examine metadata → interpret passphrase hint → use Steghide → extract concealed text → obtain the flag and next-stage keyword.

Challenge 5

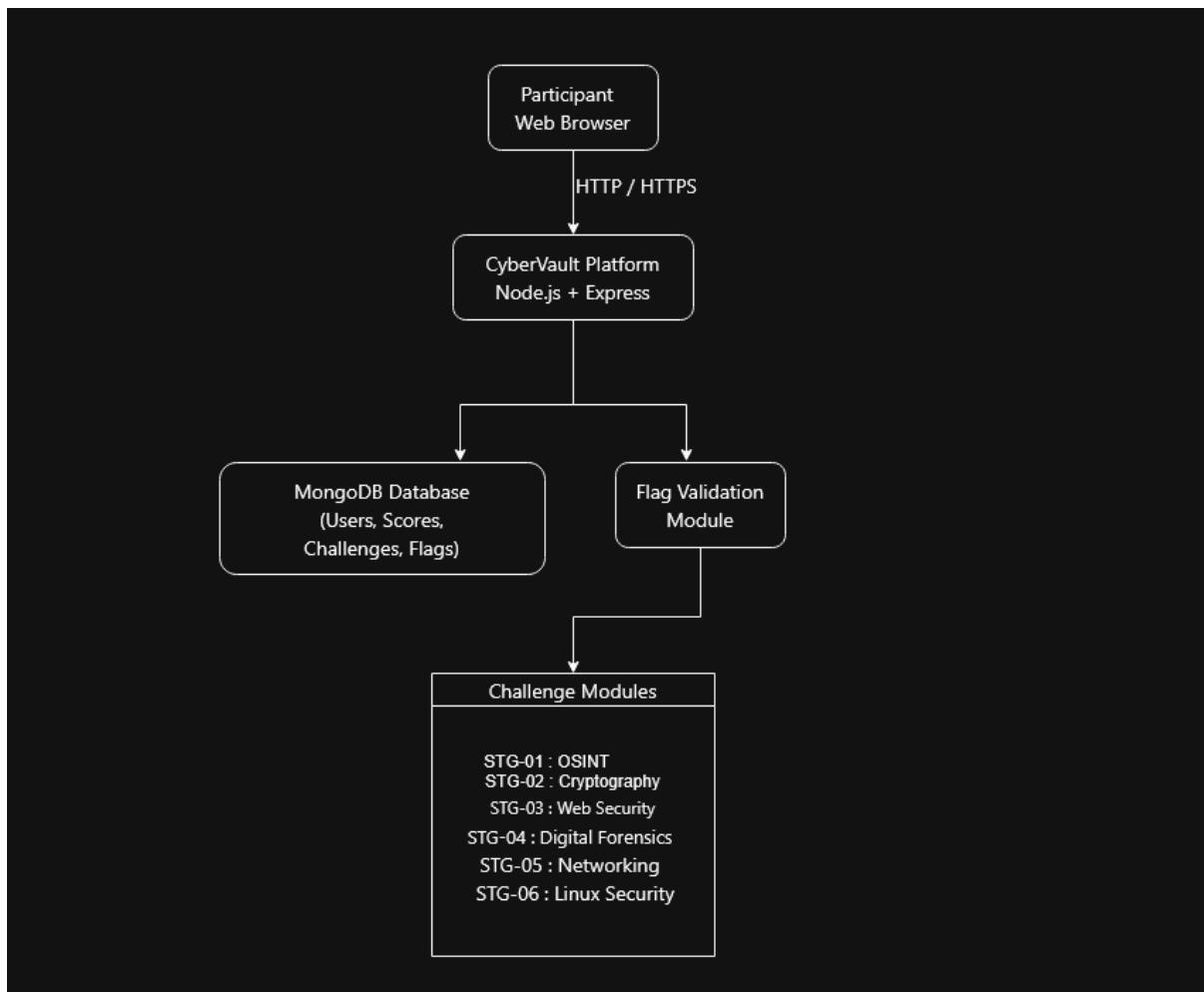
Field	Information
Stage ID &	CTF-C
Title	Broken Cipher
Domain	Cryptography
Difficulty	Moderate
Difficulty Justification	The participant must recognize that the supplied artefact contains more than one transformation layer, distinguish encoding from encryption, decode the first layer, and then use a keyword recovered from CTF-C to decrypt a classical repeating-key cipher.
Scenario	An intercepted communication is recovered during the same incident. Investigators believe the previous stage produced information necessary to reconstruct the original message.
Learning Objective	Recognize a common encoding format, distinguish encoding from encryption, apply a classical cipher decryption process, and use evidence recovered from an earlier stage.
Challenge Environment	Static text artefact distributed through the CTF platform. The file contains a Base64 representation of Vigenere ciphertext.
Player Task	Analyze the supplied text, remove the encoding layer, identify the underlying cipher, use the previous-stage keyword, recover the plaintext, and submit the flag.
Flag Location / Logic	The CTF-C flag exists only in the recovered plaintext after the intended decoding and decryption sequence. During group integration, the plaintext also provides context/access information that leads toward the Linux/System Security stage.
Required Tools	Base64 decoding utility, CyberChef or equivalent cryptographic-analysis tool, or a simple locally written Vigenere decryption script.
Dependencies	Requires the keyword recovered from CTF-C.
Expected Solution Path	Inspect intercept.txt → recognize Base64 → decode to ciphertext → identify the Vigenere-style cipher → use the CTF-C keyword → decrypt → recover plaintext and the stage flag.

Challenge 6

Field	Information
Stage ID &	CTF-LSS
Title	Permission Denied
Domain	Linux / System Security
Difficulty	Hard
Difficulty Justification	As the final Linux/System Security stage, the participant must correlate access information from earlier investigation stages with Linux identity and group membership, reason about filesystem permissions, locate a sensitive legacy backup, interpret an internal-service configuration, and use the recovered token correctly. The multi-step correlation makes the challenge appropriate as a capstone for the intended beginner-to-intermediate audience.
Scenario	The investigation leads to an isolated Linux node associated with the CyberTech Solutions incident. The supplied account appears ordinary, but the host contains a legacy backup configuration and a protected localhost service. The participant must determine why the account can access sensitive information and use that weakness to complete the investigation.
Learning Objective	Understand Linux users/groups, file ownership and permissions; identify risks caused by excessive group membership and readable backup configuration files; and investigate a local service using command-line tools.
Challenge Environment	• Simulated Linux environment provided through the CyberVault platform. • Configuration files, logs, and challenge artefacts are supplied
Player Task	• Analyze the provided Linux investigation files, identify misconfigured • permissions and exposed service information, recover the authentication, token, and obtain the final stage flag.
Flag Location / Logic	The flag is stored within the challenge artefacts and can only be recovered after identifying the exposed service configuration and correct authentication token contained within the provided Linux files.
Required Tools	Text Editor, Linux Commands, Web Browser, File Analysis Tools
Dependencies	• Requires information obtained from previous investigation stages.

	• The challenge depends on clues and tokens recovered from earlier challenges.
Expected Solution Path	Review Linux challenge files → analyze user and group information → identify exposed backup configuration → recover service token → analyze service configuration → recover the flag

PLATFORM & BOX ARCHITECTURE



The proposed architecture follows a centralized, web-based CTF model named **Cybervault**, designed for secure and scalable challenge delivery. Participants access the platform via a standard web browser using http/https, requiring no specialized client software. The core **Cybervault** platform (node.js + express) acts as the control layer, managing user sessions, challenge routing, and score tracking. A MongoDB database stores all persistent data including users, scores, challenge metadata, and flag hashes. The flag validation module operates independently to verify submitted flags against stored values, preventing direct database exposure. 6 isolated challenge modules cover OSINT, Steganography, Cryptography, Digital Forensics, Git Forensics, Reverse Engineering, Linux, Web Security Domains Programming and Scripting. Each module runs in its own isolated environment to prevent challenge interference. The platform communicates with challenge modules through internal APIs only, with no direct participant access. Network segmentation ensures participants cannot reach the database or flag validation layer directly. This layered design enforces least privilege, maintains isolation, and supports safe, repeatable challenge deployment.

TECHNOLOGY STACK

Layer	Technology
OS (Host)	Windows
Containerization	Not Required
CTF Platform	Web-Based
Web Server	Node.js / Express.js
Database	MongoDB Atlas
Database Driver	Mongoose
Programming Languages	Frontend Technologies: HTML5, CSS3, JavaScript Backend Technologies: Node.js, Express.js
Challenge Tools	ExifTool, Steghide, CyberChef, Git, Sleuthkit
Networking	HTTP, HTTPS, Localhost (Port 3000)
Monitoring/Logging	Application Logs, MongoDB Activity Logs

KEY FEATURES

- User Registration and Login
- Challenge Dashboard
- Scoreboard
- Flag Submission System
- Challenge Hints
- User Profile Management
- Progress Tracking
- Administrator Panel

DEPLOYMENT & ISOLATION PLAN

Deployment Method

- Local Server

Deployment Steps

Step 1: Install Required Software

The following software must be installed before deployment:

- Node.js Runtime Environment
- MongoDB Atlas
- Visual Studio Code
- Web Browser

Step 2: Configure Database

- Create the **CyberVault** database in MongoDB.
- Configure collections for users, challenges, scores, and flag submissions.

Step 3: Configure Backend

- Install project dependencies using npm.
- Configure MongoDB connection settings.
- Configure API routes and application settings.

Step 4: Launch the Platform

- Start the MongoDB service.
- Run the Node.js application using the Express server.
- Access the platform through: `http://localhost:3000`

Isolation Plan

- No challenge can access another challenge's filesystem.
- Participants cannot access the host or flag validation database directly.
- Read-only mounts where applicable.

Security Controls

- User authentication and authorization.
- Secure flag validation mechanism.
- Input validation to prevent malicious submissions.
- Role-based access control for administrators.
- Activity logging and monitoring.
- Database backup and recovery procedures.
- Session management for authenticated users.

Reset Mechanism

- Restore or re-download the verified master evidence image. A SHA-256 checksum can be used to verify file integrity.
- Restore or re-download the verified master copy of `intercept.txt` or image.
- Restore the original Linux challenge files and artefacts from the verified master copy.
- Replace modified files with the original challenge package to return the challenge to its initial state.

TESTING / VALIDATION PLAN

This section defines the testing and validation strategy for the **CyberVault** platform prior to full implementation. The plan verifies that each challenge functions as intended, that the flag mechanism is secure and repeatable, and that the isolation and deployment design holds under normal use.

Testing Levels

- **Unit-level challenge testing** - Each of the 6 challenges is tested independently to confirm the intended solution path produces the correct flag and that no unintended shortcut bypasses the challenge logic.
- **Integration testing** - Verifies that stage working correctly.
- **System / end-to-end testing** - A full walkthrough of the platform from registration through completion of all six stages, exercising the dashboard, flag submission system, scoreboard, and hint system together.
- **Security and isolation testing** - Confirms that challenge containers/services cannot access each other's filesystem, that the flag validation database is unreachable from the participant network segment, and that services expose only the intended ports.
- **Regression testing after reset** - Each reset/recovery procedure is re-tested to confirm the challenge returns to a solvable initial state with no residual player data.

Validation Approach by Domain

- **OSINT / Digital Forensics / Git Forensics (CTF-OSINT, CTF-DF)** - Validated by independently reproducing the intended pivot chain using only the tools listed under Required Tools, confirming no alternate artifact leaks the flag early.
- **Reverse Engineering (CTF-RE)** - Validated by testing both the legitimate win-condition path and the DevTools-console shortcut, confirming both yield the flag and that the win logic is not visible from a plain "view source".
- **Steganography (CTF-S)** - Validated by re-embedding and re-extracting the artefact with Steghide using the documented passphrase logic and confirming ExifTool alone does not reveal the passphrase.
- **Cryptography (CTF-C)** - Validated by the Base64/Vigenère chain with confirming an incorrect keyword fails cleanly rather than producing a false-positive flag.
- **Linux / System Security (CTF-LSS)** - Validated using the supplied Linux challenge artefacts and configuration files. Testing confirms that participants can recover the intended token, identify the service configuration, and successfully retrieve the final flag.

Defect Tracking and Resolution

- Issues found during testing are logged with the affected Stage ID, a description, a severity rating, and the retest outcome before a stage is marked ready.
- Any defect that would allow a flag to be recovered without following the intended solution path is treated as high severity and blocks sign-off until fixed.
- Re-tests are repeated after every fix to confirm no new issue was introduced.

Pre-Deployment Checklist

- All 6 stages solvable end-to-end using only the tools listed under Required Tools.
- Flag format (IE3132{...}) consistent across all stages and validated by the flag validation module.
- Isolation and network segmentation confirmed between challenge modules, the database, and the flag validation layer.
- Reset/recovery procedure executed and confirmed for every stage.
- Hints verified to assist without revealing the flag outright.

INDIVIDUAL CONTRIBUTIONS

Member	Role	Responsibilities
IT24102481	CTF Platform & Architecture	Architecture diagram, platform design, deployment plan, environment specification
IT24102800	Challenge 1 Challenge 2 Challenge 3	Challenge 1-3 specifications, challenge logic, tools, flags, hints.
IT24102454	Challenge 4 Challenge 5 Challenge 6	Challenge 4-6 specifications, challenge logic, tools, flags, hints.
IT24104202	Technology Stack, Testing & Documentation	Tech stack selection, testing / validation plan, risk assessment, report compilation.

RISKS, DEPENDENCIES & LIMITATIONS

Technical Risks

- **Flag leakage via metadata or source inspection** - a participant could recover a flag by inspecting file metadata, page source, or container image layers rather than solving the challenge.
Mitigation: strip unrelated metadata from distributed artefacts and review each container image before release.
- **Challenge file modification or accidental deletion** - Maintain master copies of challenge artefacts and perform integrity checks before.
Mitigation: least-privilege container users, no unnecessary bind mounts, and isolation testing before deployment.
- **Single point of failure in the flag validation module** - if the validation service is unavailable, no stage can be completed.
Mitigation: keep validation logic simple and covered by its own test cases, and log failures for quick diagnosis.

Dependency Risks

- External libraries and are third-party dependencies whose availability or version changes could affect the platform.
Mitigation: pin dependency versions and document the exact tool versions used during testing.

Implementation Risks

- Time constraints across four members with overlapping coursework could delay full implementation.
Mitigation: priorities the 6 core stages before optional/additional stages.
- Uneven familiarity with each domain across members could affect challenge quality.
Mitigation: pair review of each stage by a member outside its original designer before signing off.

Limitations of the Current Design Stage

- This report is a design and planning milestone, the platform, challenge artefacts, and Docker environments described have not yet been fully built or load tested.
- Some technological stacks are still to be finalized in the implementation phase and are marked accordingly in the Technology Stack table.
- Testing described in this report is planned rather than fully executed at this stage; it will be carried out during and after implementation, with results recorded for the final submission.

CONCLUSION

This report has presented the initial design and planning stage of **CyberVault: Project Null Byte**, a 6 stage, web-based CTF Play Box covering OSINT, digital and Git forensics, reverse engineering, steganography, cryptography, and Linux/system security. The proposed design defines a coherent investigative scenario, a progressive difficulty model from easy to expert, a centralized platform architecture with isolated challenge modules and independent flag validation, and a testing and risk plan intended to keep the eventual implementation safe, repeatable, and aligned with the module's learning outcomes.

The design has not yet been fully implemented or load-tested, and a small number of technology-stack and platform details remain to be finalized in the next phase. With the testing and validation plan and the risk mitigations set out above, the group is well positioned to move from this design stage into full implementation.

REFERENCES

- [1] OWASP Foundation, "OWASP Top 10," OWASP Foundation, 2025. [Online]. Available: <https://owasp.org/www-project-top-ten/>. [Accessed: Sep. 21, 2026].
- [2] MongoDB Inc., "MongoDB Atlas Documentation," MongoDB Inc. [Online]. Available: <https://www.mongodb.com/docs/atlas/>. [Accessed: Sep. 21, 2026].
- [3] OpenJS Foundation, "Node.js Documentation," OpenJS Foundation. [Online]. Available: <https://nodejs.org/en/docs>. [Accessed: Sep. 21, 2026].
- [4] Docker Inc., "Docker Documentation," Docker Inc. [Online]. Available: <https://docs.docker.com/>. [Accessed: Sep. 21, 2026].
- [5] The Sleuth Kit, "The Sleuth Kit and Autopsy Documentation," The Sleuth Kit. [Online]. Available: <https://www.sleuthkit.org/>. [Accessed: Sep. 21, 2026].
- [6] GCHQ, "CyberChef," GCHQ. [Online]. Available: <https://gchq.github.io/CyberChef/>. [Accessed: Sep. 21, 2026].
- [7] P. Harvey, "ExifTool Documentation," ExifTool. [Online]. Available: <https://exiftool.org/>. [Accessed: Sep. 21, 2026].
- [8] SLIIT, "IE3132 Penetration Testing – Assignment 01 Brief: CTF Play Box Implementation-Project Initiation," SLIIT, 2026.

- [9] OSINT Framework, "OSINT Framework." [Online].
Available: <https://osintframework.com/>. [Accessed: Sep. 21, 2026].
- [10] IBM, "What is digital forensics?," IBM. [Online].
Available: <https://www.ibm.com/think/topics/digital-forensics>. [Accessed: Sep. 21, 2026].
- [11] Siemens, "Reverse Engineering," Siemens. [Online].
Available: <https://www.siemens.com/en-us/technology/reverse-engineering/>. [Accessed: Sep. 21, 2026].
- [12] Advanced Enumerated OSINT Framework for Cybersecurity and Digital Forensics, in Proc. IEEE Int. Conf. Cyber Security and Digital Forensics, Boracay Island, Philippines, Feb. 2026, pp. 1-6. [Online]. Available: <https://ieeexplore.ieee.org/document/11467741>
- [13] A Guided Modeling Approach for Secure System Design, in Proc. IEEE Int. Conf. Software Architecture, Hyderabad, India, Jun. 2024, pp. 1-8. [Online].
Available: <https://ieeexplore.ieee.org/document/10628381>
- [14] Redefining OSINT Software Architecture With System-Centric Architecture Design: A Framework Shaped by QAW, ADD, and ATAM, in Proc. IEEE Int. Conf. Software Engineering, 2025. [Online]. Available: <https://ieeexplore.ieee.org/document/10971222>
- [15] Design and Development of Security Services Using Security Patterns and Microservices Security Practices, in Proc. IEEE Int. Conf. Software Security, Bangkok, Thailand, Jan. 2025. [Online]. Available: <https://ieeexplore.ieee.org/document/11196387>
- [16] Integrating Security into Industrial Control System Architecture Based on IEC 42010, in Proc. IEEE Int. Conf. Industrial Cyber-Physical Systems, Sep. 2024. [Online].
Available: <https://ieeexplore.ieee.org/abstract/document/10710929>

APPENDICES

AI Prompts

- Can you give me information on architecture design? I'd like to understand what it involves, its key principles, and how it's applied in practice.
- Can you give me information on how to build OSINT? I'd like to understand the tools, techniques, and steps involved in setting up an open-source intelligence capability.
- Road map to build OSINT, Digital Forensics, Git Forensics, Steganography combined CTF.
- Using Digital Forensics, Steganography, OSINT how to create a separate CTF give me the road map.
- How to create a CTF using Reverse Engineering, Web Security, Programming / Scripting give me the roadmap.
- Create a complete, self-contained HTML/JavaScript CTF challenge game "ghost_runner.html" with the following specifications:
- Create a table structure only to show how to add them for a document.
- Analyze the IE3132 Penetration Testing Assignment 01 - CTF Play Box Implementation - Project Initiation.
- I am responsible for the Challenge Design section of the group CTF. According to the assignment, identify exactly what information must be documented for each stage, including Stage ID and title, domain, difficulty and justification, scenario, learning objective, challenge environment, player task, flag logic, required tools, dependencies, expected solution path, validation, hints, and reset/recovery.

- Also explain how my individual contribution should fit into the complete six-stage group CTF report. Do not invent assignment requirements that are not present in the provided brief.
- Guide me step by step to create an Easy Steganography CTF called Hidden in Plain Sight using ExifTool and Steghide.
- My Steghide challenge is not working correctly. Check this terminal output and tell me how to fix it.
- Help me create a Moderate Cryptography CTF called Broken Cipher using Base64 and Vigenere cipher. It should use a key from the previous challenge.
- Guide me to test Broken Cipher from a clean player folder and verify that the flag cannot be found without the correct key.
- Create a testing and validation plan plus a private walkthrough for Hidden in Plain Sight, Broken Cipher and Permission Denied, including the expected solving steps and evidence I should capture.